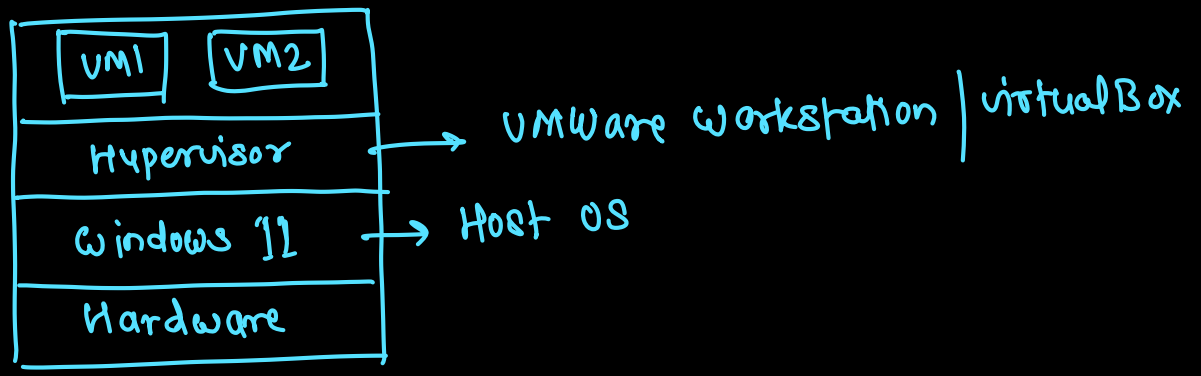
Hyper-V

What is virtualization

- Virtualization is the creation of a virtual -- rather than actual -- version of something, such as an operating system (OS), a server, a storage device or network resources
- Virtualization uses software that simulates hardware functionality in order to create a virtual system → Hypervisor
- This practice allows IT organizations to operate multiple operating systems, more than one virtual system and various applications on a single server
- Types
  - Network virtualization
  - Storage virtualization → LVM | iSCSI
  - Data virtualization
  - Desktop virtualization
  - Application virtualization
  - Hardware virtualization
  - Operating System virtualization
    - → containerization

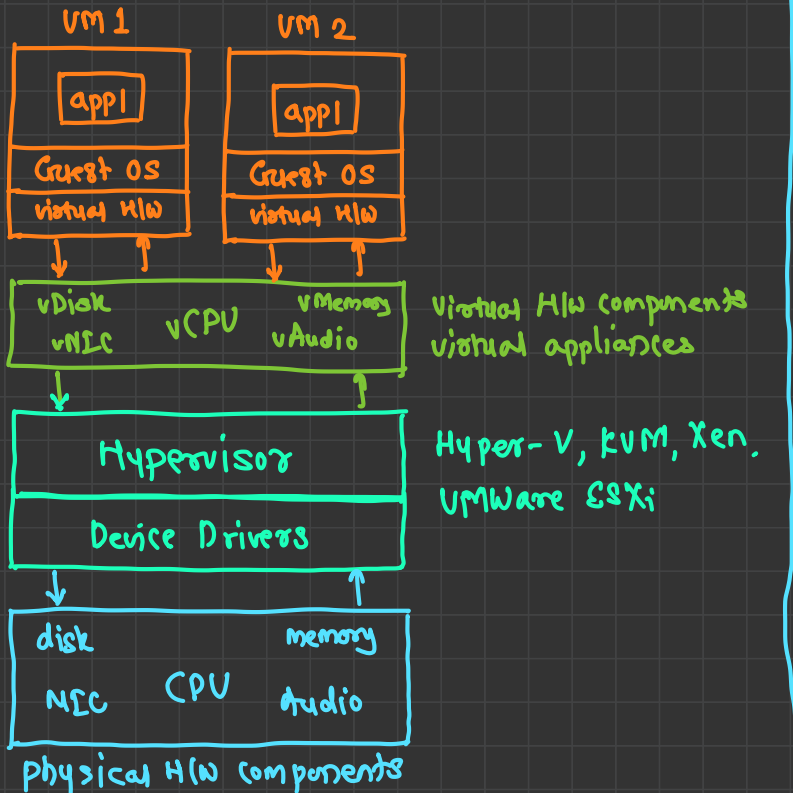


Hardware Virtualization

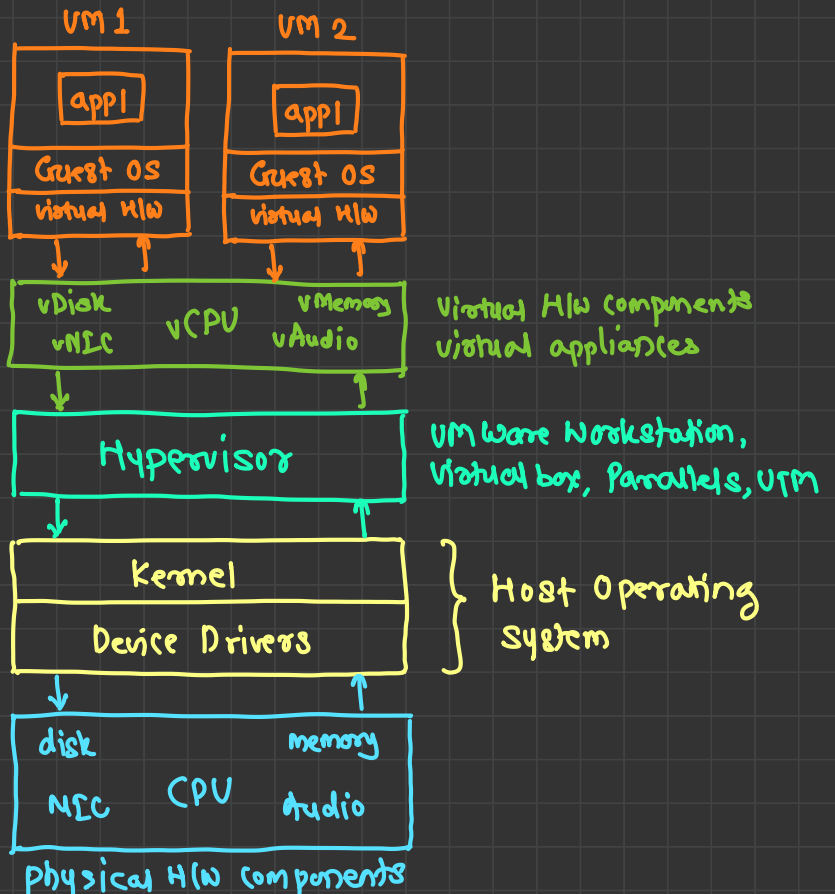


- Hardware virtualization is the process of creating a **virtual version of a physical computing environment** using a layer of software called a **hypervisor**
- Instead of running directly on physical hardware, multiple **virtual machines (VMs)** can share the same physical resources (CPU, memory, storage, network). Each VM behaves like an independent computer with its own operating system and applications
- **Hypervisor (Virtual Machine Monitor – VMM)**
  - The software layer that enables virtualization.
  - Types:
    - Type 1 (Bare-metal): Runs directly on hardware (e.g., VMware ESXi, Microsoft Hyper-V, Xen)
    - Type 2 (Hosted): Runs on top of a host OS (e.g., VirtualBox, VMware Workstation)
- **Virtual Machine (VM)**
  - A software-based emulation of a physical computer
  - Each VM has its own virtual CPU, memory, disk, and network interface
- **Hardware Support**
  - Modern CPUs (Intel VT-x, AMD-V) and hardware (IOMMU, SR-IOV for networking) include extensions to make virtualization efficient and secure

Type I → Bare Metal



Type II - Hosted



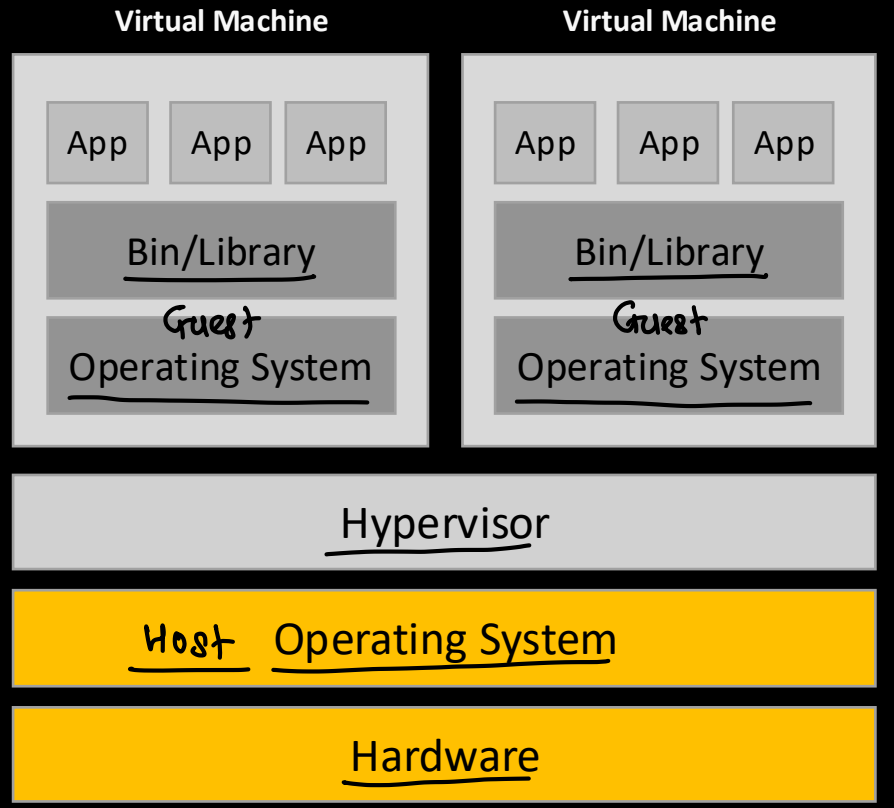
Types of hardware virtualization



- Type I → used by cloud providers
  - A Type 1 hypervisor runs directly on the host machine's physical hardware, and it's referred to as a bare-metal hypervisor
  - It doesn't have to load an underlying OS first
  - With direct access to the underlying hardware and no other software, it is more efficient and provides better performance
  - It is best suited for enterprise computing or data centers
  - E.g. VMware ESXi, Microsoft Hyper-V server and open source KVM
- Type II → used by developers | testers | End users
  - A Type 2 hypervisor is typically installed on top of an existing OS, and it's called a hosted hypervisor → Host OS
  - It relies on the host machine's pre-existing OS to manage calls to CPU, memory, storage and network resources
  - E.g. VMware Fusion, Oracle VM VirtualBox, Oracle VM Server for x86, Oracle Solaris Zones, Parallels and VMware Workstation

Virtualized Deployment

- It allows you to run multiple Virtual Machines (VMs) on a single physical server's CPU
- Virtualization allows applications to be isolated between VMs and provides a level of security as the information of one application cannot be freely accessed by another application
- Virtualization allows better utilization of resources in a physical server and allows better scalability because
  - an application can be added or updated easily
  - reduces hardware costs
- With virtualization you can present a set of physical resources as a cluster of disposable virtual machines
- Each VM is a full machine running all the components, including its own operating system, on top of the virtualized hardware



Advantages of virtualization



- **Lower costs**
  - Virtualization reduces the amount of hardware servers necessary within a company and data center
  - This lowers the overall cost of buying and maintaining large amounts of hardware
- **Easier disaster recovery**
  - Disaster recovery is very simple in a virtualized environment
  - Regular snapshots provide up-to-date data, allowing virtual machines to be feasibly backed up and recovered
  - Even in an emergency, a virtual machine can be migrated to a new location within minutes
- **Easier testing**
  - Testing is less complicated in a virtual environment
  - Even if a large mistake is made, the test does not need to stop and go back to the beginning
  - It can simply return to the previous snapshot and proceed with the test
- **Quicker backups**
  - Backups can be taken of both the virtual server and the virtual machine
  - Automatic snapshots are taken throughout the day to guarantee that all data is up-to-date
  - Furthermore, the virtual machines can be easily migrated between each other and efficiently redeployed
- **Improved productivity**
  - Fewer physical resources results in less time spent managing and maintaining the servers
  - Tasks that can take days or weeks in a physical environment can be done in minutes
  - This allows staff members to spend majority of their time on more productive tasks, like raising revenue and fostering business initiatives

Hyper-V

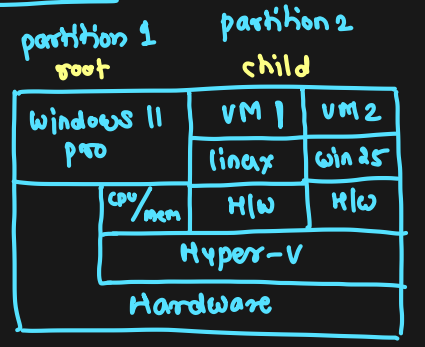
Introduction



- Hyper-V is Microsoft's hardware virtualization technology for x64 versions of Windows
- It allows multiple isolated operating systems to run simultaneously on a single physical hardware platform
- Hyper-V is a **Type 1 (bare-metal) hypervisor**, meaning it runs directly on the hardware rather than on top of a host operating system
- **System Requirements**
  - Before deploying Hyper-V, verify the following hardware requirements:
  - 64-bit processor with **SLAT (Second Level Address Translation)**
  - Processor supporting hardware-assisted virtualization (**Intel VT** or **AMD-V**)
  - Processor supporting **VM Monitor Mode extensions**
  - Enough memory for both the host and guest VMs
  - Minimum 4 GB RAM (8 GB or more recommended)

Core Architecture

- Hyper-V architecture is built around the concept of **partitions**
- A partition is a **logical unit of isolation**, supported by the hypervisor, in which operating systems execute
- **Root (Parent) Partition**
  - The root partition is the first partition created when the hypervisor starts
  - It runs the Windows operating system
  - It is responsible for:
    - Running the virtualization management stack and having **direct access to hardware devices**
    - Creating child partitions via the **Hypercall API**
    - Managing hardware devices (except **processor scheduling** and **physical memory allocation**, which are handled by the hypervisor)
    - Handling power management, Plug and Play operations, and recording hardware failure events
- **Child Partition**
  - Child partitions ~~host~~ guest operating systems
  - A child partition does **not have direct access to physical hardware resources**; instead, it receives a virtual view of the resources
  - Requests for virtual devices are redirected to devices in the root partition for processing via the **VMBus** or the hypervisor



Technical Features



- **Live Migration:** Allows a running VM to be moved from one physical host to another with **no downtime or disruption of connectivity**. Supports simultaneous migration, compression, and SMB transport options
- **Hyper-V Replica:** Asynchronously replicates VMs from a primary site to a replica site over a standard IP network, with configurable replication frequencies (e.g., 5, 10, 15 minutes). In a disaster, it enables planned or unplanned failover
- **Checkpoints:** Captures the **state, data, and hardware configuration** of a running VM at a specific point in time, allowing the VM to be easily reverted to a previous state. Supports standard checkpoints and production checkpoints (which use VSS to create data-consistent backups)
- **Failover Clustering:** Combined with Windows Server Failover Clustering to provide **high availability**. If a physical host fails, the VMs running on it automatically restart on another healthy host in the cluster
- **Virtual Storage:** Uses the **VHDX** virtual hard disk format, supporting disks up to **64 TB** with resilience to data corruption. Also supports pass-through disks and Storage Spaces Direct integration
- **Security Features**
  - **Secure Boot:** Verifies digital signatures of boot files to prevent malware
  - **Shielded VMs:** Encrypted virtual disks that can only run on specific guarded hosts
  - Guests cannot communicate with each other (except through traditional networks) and cannot perform DMA attacks

Supported Guest Operating Systems

- Hyper-V supports the following guest operating systems:
- All supported Windows versions
- Linux distributions: Red Hat Enterprise Linux, Debian, Oracle Linux, SUSE, Ubuntu
- FreeBSD
- Microsoft only provides support for issues found in Microsoft operating systems and Integration Services

Typical Use Cases



- **Server Consolidation:** Consolidate multiple physical servers onto fewer, more powerful hosts, saving space and energy
- **Development and Test Environments:** Quickly provision and restore virtualized test environments without affecting production systems
- **VDI (Virtual Desktop Infrastructure):** Combine with Remote Desktop Virtualization for centralized desktop management
- **Private Cloud Infrastructure:** Integrate with Storage Spaces Direct and SDN to run on-premises virtualized workloads
- **Disaster Recovery and Business Continuity:** Use live migration and replica features to minimize planned and unplanned downtime

Hyper-V vs VMWare

hypervisor for windows (pro onwards)

customized Linux distro

Aspect	Hyper-V	VMware ESXi
Hypervisor Design	Microkernelized: Device drivers run in the root/parent partition, not in the hypervisor itself	Monolithic: Device drivers are an integral part of the hypervisor layer
Driver Location	Root partition (Windows OS)	Hypervisor layer
Initialization	Faster, as drivers load in the OS layer	Slower, as drivers initialize in the hypervisor layer
Hardware Compatibility	Broad, since drivers are standard Windows drivers → shared with root partition	Limited to VMware's Hardware Compatibility List (HCL)
Security Patching	Root partition requires OS patches	Hypervisor layer is minimal and requires fewer patches

When to choose

- **Choose Hyper-V If**
  - Your organization is **Microsoft-centric** (Windows Server, Active Directory, System Center, Azure)
  - You want **lower licensing costs** and a predictable CAPEX model
  - You need **tight Azure hybrid cloud integration**
  - Your team is already skilled in **PowerShell and Windows administration**
  - You primarily run **Windows workloads**
- **Choose VMware If**
  - You need **maximum scalability** (thousands of VMs, very high-density clusters)
  - You run **heterogeneous workloads** (mixed Linux/Windows, legacy OS support, macOS in some cases)
  - You require **advanced networking** (NSX micro-segmentation)
  - You need **true Fault Tolerance** for zero-downtime critical applications
  - You have the **budget** to absorb **Broadcom**'s subscription costs
  - You want the **most mature enterprise ecosystem** and third-party integrations

Create VM

- **Open Hyper-V Manager:** Search for "Hyper-V Manager" in your Start menu and open it
- **Start the Wizard:** In the right-hand "Actions" pane, click **New > Virtual Machine**
- **Specify Name and Location:** Give your VM a descriptive name. You can optionally choose a custom folder to store its configuration files
- **Specify Generation:** Choose **Generation 2** unless you need to run a legacy OS that requires BIOS. Generation 2 offers UEFI and Secure Boot.
- **Assign Memory:** Allocate the startup RAM. For Windows 11, 4GB (4096 MB) is a good minimum. You can also enable **Dynamic Memory** to let Hyper-V adjust RAM usage automatically
- **Configure Networking:** Select a virtual switch to connect the VM to your network. If you haven't created one, you can skip this and configure it later
- **Connect Virtual Hard Disk:** This is the key storage configuration step
  - **Create a virtual hard disk:** This is the default. You can name the disk, choose its location, and set its size
  - **Use an existing virtual hard disk:** Select this if you already have a .vhd or .vhdx file
  - **Attach a virtual hard disk later:** You can add a disk after the VM is created
- **Installation Options:** Select **Install an operating system from a bootable image file** and browse to your OS .iso file.
- **Finish:** Review the summary and click **Finish**